



Cross-Platform Real-Time Audio Signal Analysis and High-Frequency Noise Filtering CLI Tool

Adil Kaan Gd
Muhammet Taha Demir
Computer Engineering

April 21, 2026

Abstract

This study focuses on the development of a real-time audio processing tool designed to eliminate high frequency disturbances from live microphone feeds. The project aims to provide a Command Line Interface (CLI) solution that utilizes Digital Signal Processing (DSP) techniques to improve audio clarity in professional and daily communication. The primary mechanism involves capturing raw audio data, converting it into the frequency domain via Fast Fourier Transform (FFT), and applying a user-defined low-pass filter to suppress unwanted noise such as hissing, fan sounds, and electrical interference. Developed using Python and core libraries like PyAudio and NumPy, the tool ensures cross-platform compatibility and low-latency performance. This report details the system architecture, mathematical foundations, and the implementation of a modular filtering framework.



1 Introduction

Digital communication and high-quality audio recording have become essential in modern professional environments, yet ambient noise remains a significant challenge. Consumer-grade microphones often capture a wide range of high-frequency background noises, including computer fan whirring, coil whine from electronic components, and persistent atmospheric hissing. These disturbances not only degrade the quality of voice recordings but also reduce the intelligibility of speech during live transmissions.

The objective of this project is to develop a lightweight, efficient command-line tool that acts as a real-time low-pass filter. By providing users with direct control over the "cut-off frequency," the tool allows for the precise removal of noise components that reside above the human vocal range. Unlike complex and resource-heavy audio suites, this CLI-based approach prioritizes speed, modularity, and ease of use across different operating systems like Windows, macOS, and Linux. The project seeks to bridge the gap between complex signal processing theory and practical, everyday utility for clearer digital audio.

2 Procedure

The tool relies on fundamental signal processing principles. The steps are as follows:

1. Audio Capture and Preprocessing: Real-time audio is collected from the microphone, normalized, and framed.
2. Frequency Domain Conversion: The Fast Fourier Transform (FFT) is applied to the audio frames to extract frequency components.
3. Low-Pass Filtering: A cut-off frequency (provided by the user via CLI) is applied. Frequencies above this threshold are zeroed out.
4. Time Domain Reversion: The filtered spectrum is converted back to an audio signal using the Inverse Fast Fourier Transform (IFFT) and sent directly to the speakers.
5. CLI Development: The tool is written in Python (using PyAudio, SciPy, and NumPy) to ensure cross-platform compatibility across Windows, macOS, and Linux.

3 Theory

The project relies on analyzing sampled audio data points. The Fast Fourier Transform (FFT) is used to map the signal into its frequency components. A low-pass filter is then utilized to allow frequencies below a certain threshold to pass, while blocking the higher frequencies where noise like crackling and hissing typically resides.

- Sampling: According to the Nyquist theorem, the sampling rate (F_s) must be at least twice the highest frequency present in the signal to capture it accurately.
- FFT: The transformation is expressed as $X[k] = \sum [n] * e^{j*2\pi*k*n/N}$. High-frequency noises appear as large amplitudes at higher k values, and the filter zeros these out.

4 Results

The expected outcomes of this project include:

- A module that performs real-time frequency analysis from microphone input.

- An effective low-pass filtering mechanism that instantly removes high-frequency noise based on a command-line parameter.
- A fully functional, cross-platform terminal application.

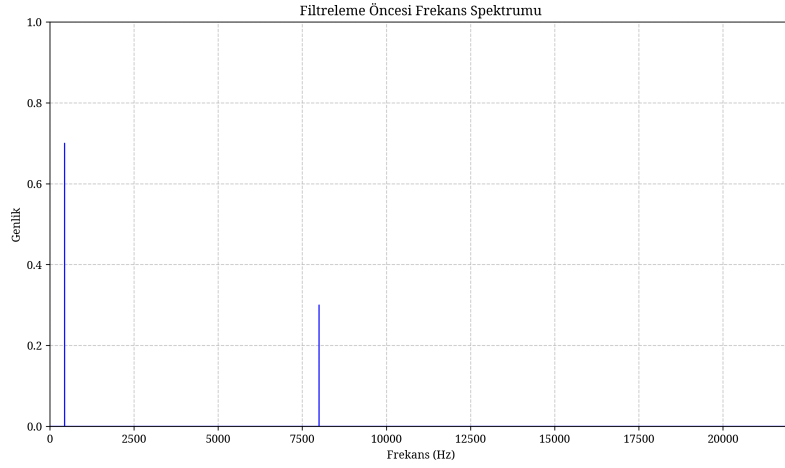


Figure 1: Frequency spectrum before filtering

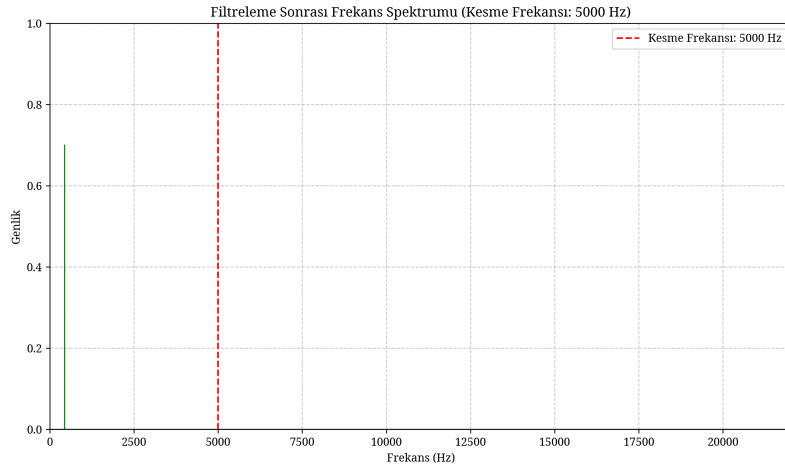


Figure 2: Frequency spectrum after filtering

5 Remarks and Recommendations

Possible improvements for future iterations:

- Implementing more advanced digital filters like Butterworth or Chebyshev for smoother frequency cut-offs.

- Adding a dynamic cut-off frequency feature that automatically adjusts based on ambient noise analysis.
- Optimizing the FFT block sizes to minimize processing latency.
- Adding CLI arguments to let the user select specific input/output audio devices.

References

- [1] Oppenheim, A. V., & Schafer, R. W. (1989). Discrete-time signal processing. Prentice Hall.
- [2] Lyons, R. G. (2011). Understanding digital signal processing. Prentice Hall.
- [3] Smith, S. W. (1997). The scientist and engineer's guide to digital signal processing. California Technical Publishing.
- [4] PyAudio library documentation: <https://people.csail.mit.edu/hubert/pyaudio/>
- [5] SciPy library documentation: <https://docs.scipy.org/doc/scipy/reference/>
- [6] NumPy library documentation: <https://numpy.org/doc/stable/>
- [7] Argparse library documentation: <https://docs.python.org/3/library/argparse.html>